



南开大学
Nankai University

《软件测试与维护》实验报告

(2025~2026 学年第二学期)

实验名称：微服务系统运维实验：监控部署与混沌工程

学 院：软件学院

姓 名：李宗杭

学 号：2311451

指导老师：张圣林

2026 年 5 月 13 日

目录

1 实验背景	1
2 实验内容	1
3 实验步骤	2
3.1 Prometheus+Grafana 监控部署	2
3.2 ChaosMesh 故障注入	5
4 实验结果与分析	8
4.1 Prometheus 抓取的数据	8
4.2 Grafana 仪表盘展示	9
5 实验总结	9

实验 2：基于 Prometheus+Grafana 的监控系统部署与 ChaosMesh 混沌工程实验

1 实验背景

运维是确保软件系统稳定、可靠运行的关键环节。随着现代软件系统复杂度的增加，运维更加注重实时监控、性能优化和故障排查。Prometheus 和 Grafana 是目前广泛使用的监控与可视化工具：Prometheus 采用拉取式数据模型高效收集指标，Grafana 提供丰富的图表展示。同时，混沌工程通过主动注入故障（如 Pod 销毁、网络延迟等）验证系统的鲁棒性和容错能力。ChaosMesh 是面向 Kubernetes 的混沌工程工具，可配合监控体系观测故障发生、传播与恢复的全流程。本次实验以 SockShop 微服务为载体，部署 Prometheus+Grafana 监控体系，并使用 ChaosMesh 注入故障，完成“监控搭建—可视化—故障注入—指标分析”的全流程实践。

2 实验内容

本次实验在已部署 SockShop 的 Minikube 集群上完成以下任务：

1. 部署 Prometheus+Grafana 监控系统，采集 SockShop 各微服务的运行指标；
2. 配置 Grafana 数据源，导入仪表盘（Dashboard）并可视化监控数据；
3. 安装 Helm 包管理工具，通过 Helm 部署 ChaosMesh；
4. 使用 ChaosMesh 定义故障实验，注入故障；
5. 观察故障前后 Grafana 监控指标的变化，分析系统在异常下的表现。

3 实验步骤

3.1 Prometheus+Grafana 监控部署

首先，进入 SockShop 项目目录，使用 kubectl 创建 monitoring 命名空间下的所有监控资源：

```
1 kubectl create -f ./deploy/kubernetes/manifests - monitoring
```

查看监控服务（Prometheus 和 Grafana 均为 NodePort 类型）：

```
1 kubectl get svc -n monitoring
```

```
PS C:\Users\李尔里德> kubectl get svc -n monitoring
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
grafana              NodePort    10.108.26.47  <none>         80:31300/TCP     43h
kube-state-metrics  ClusterIP   None          <none>         8080/TCP,8081/TCP 43h
node-exporter        ClusterIP   None          <none>         9100/TCP         43h
prometheus           NodePort    10.96.154.58  <none>         9090:31090/TCP   43h
```

图 1: monitoring 服务列表

通过 minikube 获取访问 URL：

```
1 minikube service prometheus -n monitoring --url
2 minikube service grafana -n monitoring --url
```

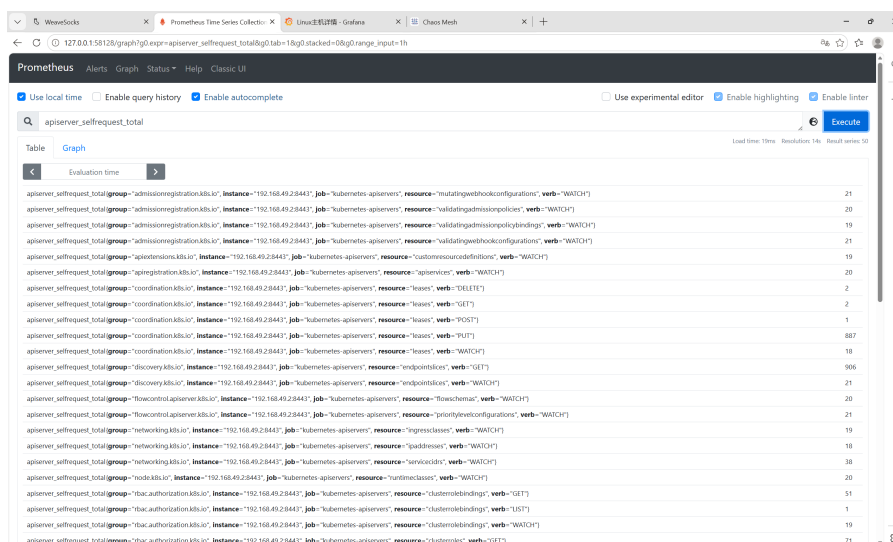
```
PS C:\Users\李尔里德> minikube service prometheus -n monitoring --url
http://127.0.0.1:58128
```

图 2: 获取 prometheus 的 URL

```
PS C:\Users\李尔里德> minikube service grafana -n monitoring --url
http://127.0.0.1:64050
```

图 3: 获取 grafana 的 URL

在浏览器中打开上述 URL，分别进入 Prometheus 和 Grafana 界面：



The screenshot shows the Prometheus web interface with a query for 'apiserver_selfrequest_total'. The interface includes a search bar, a 'Table' tab, and a 'Graph' tab. The 'Table' tab is active, displaying a list of metrics with columns for group, instance, job, resource, verb, and value. The metrics are sorted by value in descending order.

group	instance	job	resource	verb	value
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="mutatingwebhookconfigurations"	verb="WATCH"	21
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="validatingadmissionpolicies"	verb="WATCH"	20
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="validatingadmissionpolicybindings"	verb="WATCH"	19
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="validatingwebhookconfigurations"	verb="WATCH"	21
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="customresourcedefinitions"	verb="WATCH"	19
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="apiservices"	verb="WATCH"	20
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="leases"	verb="DELETE"	2
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="leases"	verb="GET"	2
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="leases"	verb="POST"	1
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="leases"	verb="PUT"	887
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="leases"	verb="WATCH"	18
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="endpointslices"	verb="GET"	906
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="endpointslices"	verb="WATCH"	21
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="flowschemas"	verb="WATCH"	20
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="priorityeventconfigurations"	verb="WATCH"	21
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="ingresses"	verb="WATCH"	19
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="ipaddresses"	verb="WATCH"	18
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="services"	verb="WATCH"	38
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="runtimeclasses"	verb="WATCH"	20
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="clusterrolebindings"	verb="GET"	51
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="clusterrolebindings"	verb="LIST"	1
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="clusterrolebindings"	verb="WATCH"	19
apiserver_selfrequest_total	192.168.49.28443	job="kubernetes-apiservers"	resource="clusterrolebindings"	verb="GET"	71

图 4: Prometheus 页面

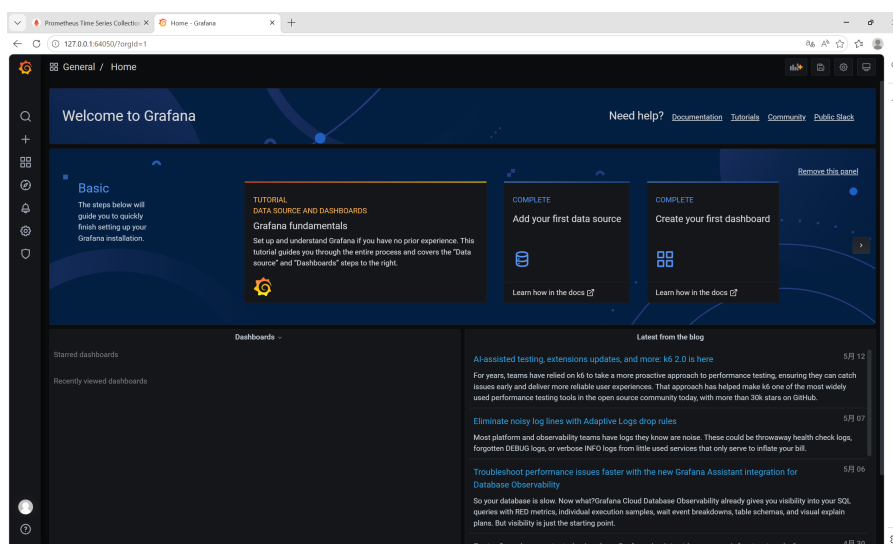


图 5: Grafana 页面

登录 Grafana 后，添加 Prometheus 数据源：

- 进入 Configuration → Data Sources → Add data source；
- 选择 Prometheus，填写 Prometheus 的 URL 地址；
- 点击 “Save & Test”。

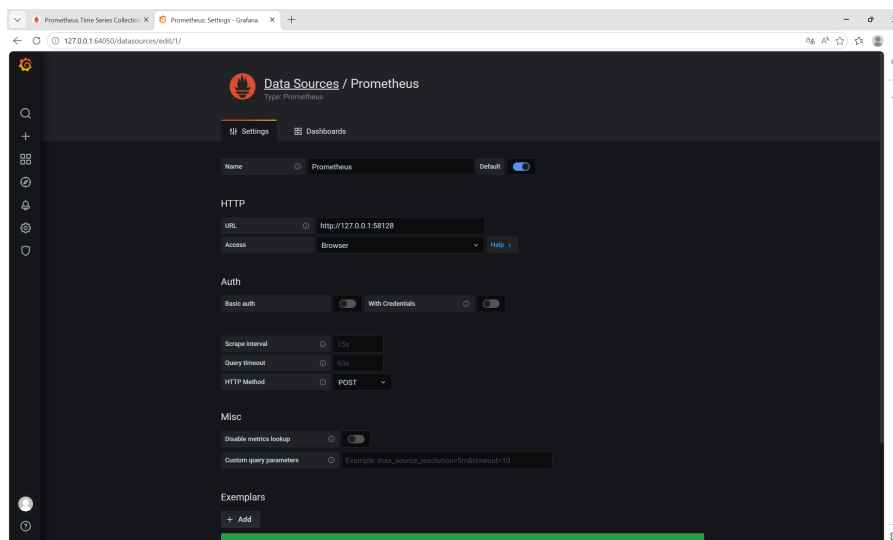


图 6: 添加 Prometheus 数据源

导入预定义的仪表盘 (Dashboard):

- 点击 “Create” → “Import”;
- 输入仪表盘 ID;
- 选择 Prometheus 数据源, 点击 Import。

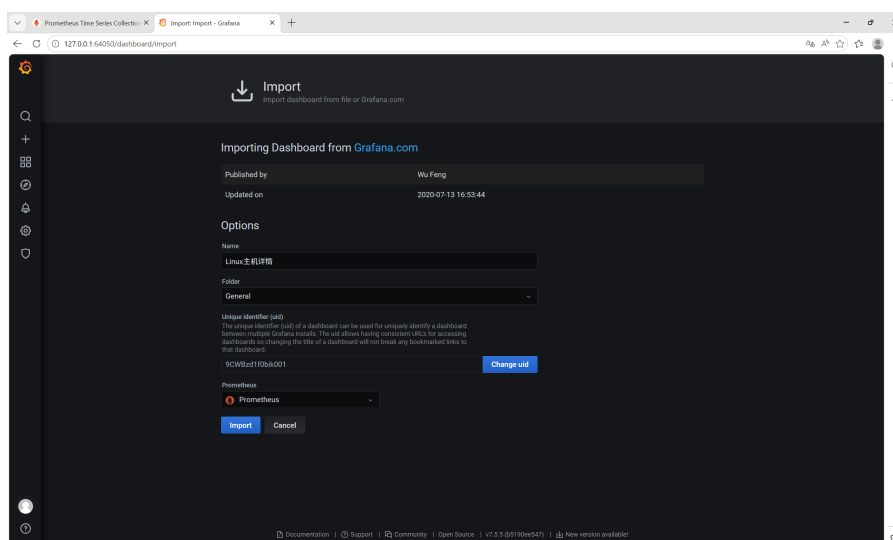


图 7: 创建仪表盘

成功导入后, Grafana 将展示 SockShop 各微服务的 CPU、内存、请求速率等监控图表。

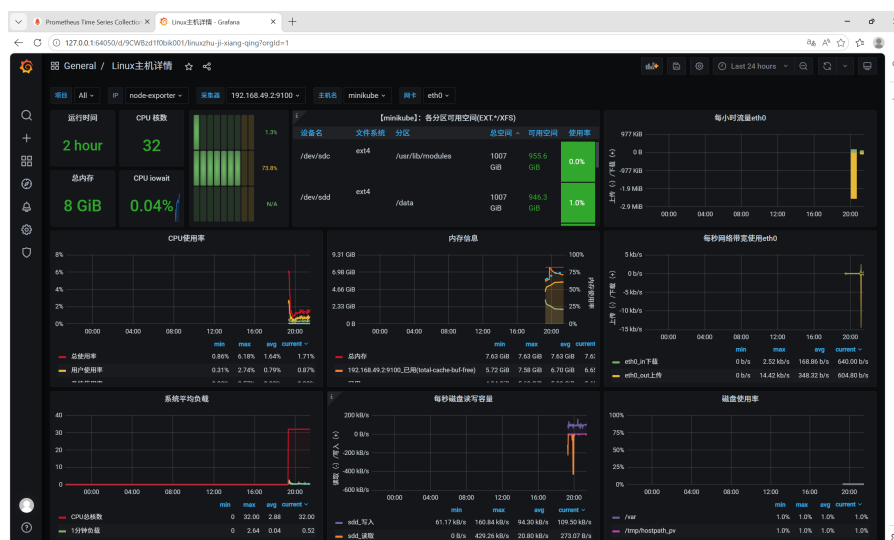


图 8: 仪表盘页面

3.2 ChaosMesh 故障注入

安装 Helm

下载 Helm 压缩包，解压并将 helm.exe 所在目录加入系统 Path 环境变量。验证安装：

```
1 helm version
```

```
PS C:\Users\李尔里德> helm version
version.BuildInfo{Version:"v4.1.4", GitCommit:"85fa37973dc9e42b76e1d2883494c87174b6074f", GitTreeState:"clean", GoVersion:"go1.25.9", KubeClientVersion:"v1.35"}
```

图 9: Helm 版本信息截图

安装 ChaosMesh

添加 ChaosMesh Helm 仓库并部署：

```
1 helm repo add chaos-mesh https://charts.chaos-mesh.org
2 helm install chaos-mesh chaos-mesh/chaos-mesh --namespace ...
   chaos-testing --create-namespace
```

安装完成后，通过端口转发打开 ChaosMesh Dashboard：

```
1 kubectl port-forward -n chaos-testing svc/chaos-dashboard 2333:2333
```

在浏览器访问 localhost:2333:

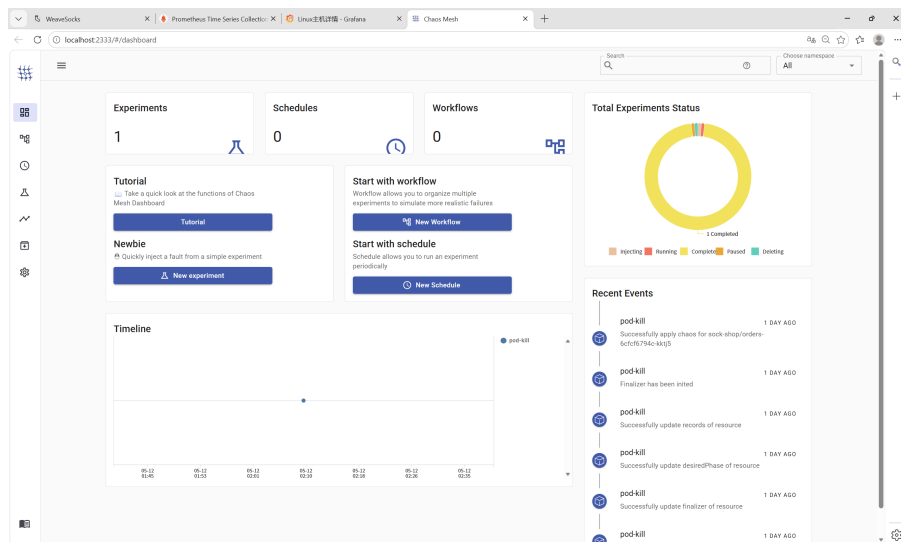


图 10: ChaosMesh 页面

创建故障实验

通过 ChaosMesh 的 UI 来注入故障并执行

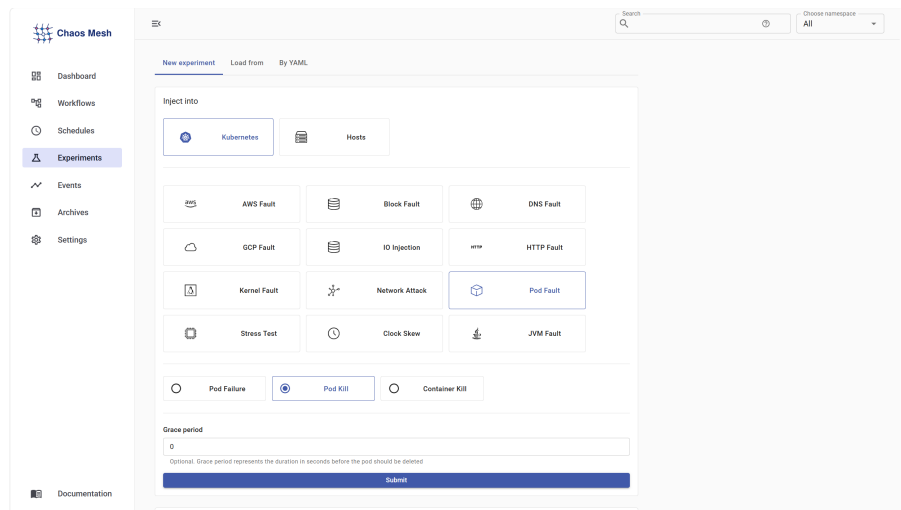


图 11: 创建 pod-kill 故障-1

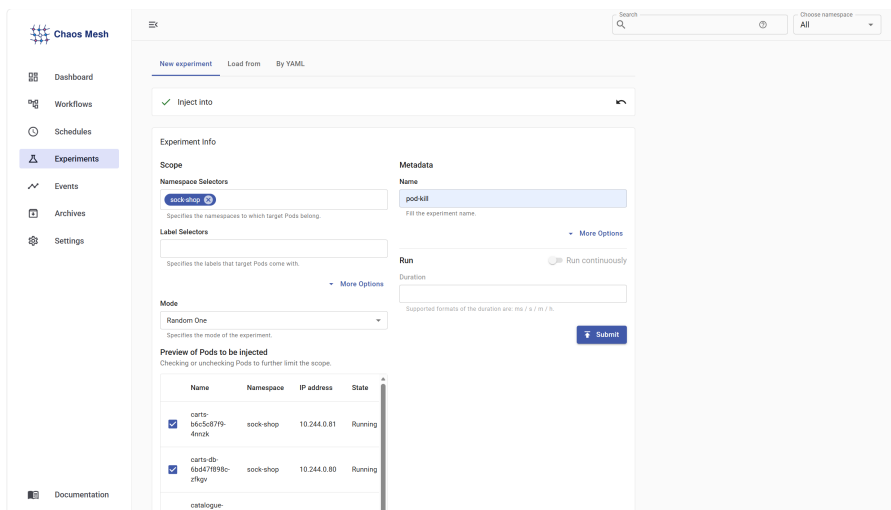


图 12: 创建 pod-kill 故障-2

查看实验状态

在 UI 的“Experiment”部分可看到实验状态变为“Completed”，表示故障已成功注入，在 23:45 杀死了名为 user-db-5b8fd7455-rfd4g 的 Pod。

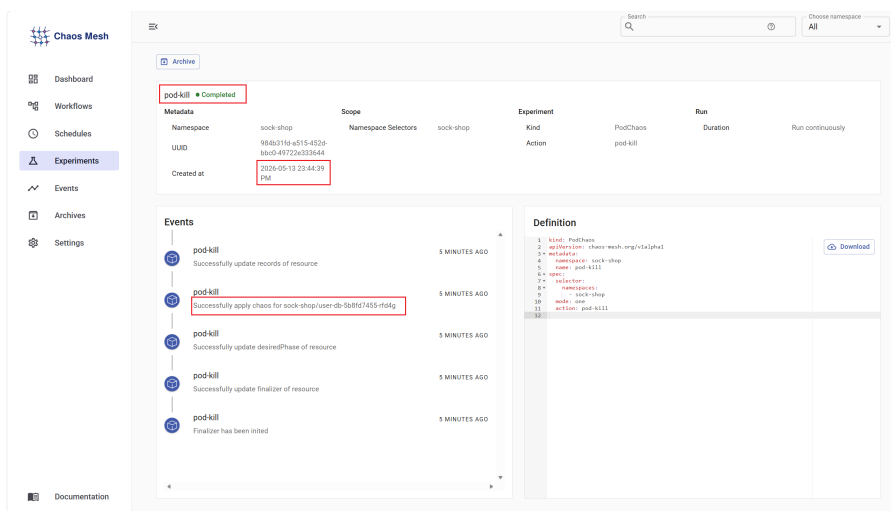


图 13: 故障注入成功

4 实验结果与分析

4.1 Prometheus 抓取的数据

通过 Prometheus 的 Graph 界面可以查询指标，通过下面这个语句查询 sock-shop 命名空间下所有 Pod 名称以 user-db-开头的 Pod 的阶段状态：

```
1 kube_pod_status_phase{namespace="sock-shop", pod=~"user-db-.*"}
```

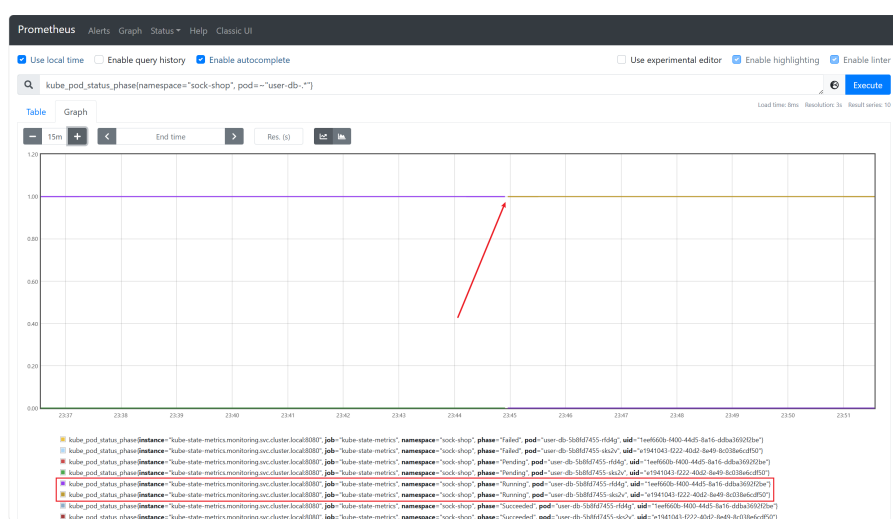


图 14: Pod 各阶段状态指标

在 23:45 时，“Pod 名为 user-db-5b8fd7455-rfd4g 且处于 Running 态”这个指标从 1 变成了 0，也就是被注入的故障所杀死的 Pod，短暂延迟后，“Pod 名为 user-db-5b8fd7455-sks2v 且处于 Running 态”这个指标从 0 变成了 1，这是 Kubernetes 的 Deployment 控制器检测到 Pod 缺失，立即创建了一个新的 Pod。

4.2 Grafana 仪表盘展示



图 15: 故障注入后的仪表盘

故障注入后，Kubernetes Deployment 控制器立即触发了自愈机制，旧 Pod 被删除的同时，新 Pod 开始创建并启动。在约 23:45 的启动阶段，节点磁盘出现了显著的读写尖峰，这是数据库初始化、加载数据和写入日志的典型行为。

5 实验总结

通过本次实验，深入理解了云原生环境下监控与混沌工程的配合价值。Prometheus+Grafana 提供了从数据采集到可视化展示的完整链路，而 ChaosMesh 以声明式方式注入故障，能够安全地验证系统的容错边界。实验过程中观察到故障注入瞬时引起的指标波动，以及 Kubernetes 自愈机制如何快速恢复服务，这让我认识到：良好的可观测性是保障微服务稳定性的基础，混沌测试则是主动发现薄弱环节的有效手段。未来可以进一步探索网络延迟、CPU 压力等更复杂的故障类型。